

Application Flow Document (App Flow)

Project: Tracker_Project (Autonomous Competitor Tracker)
Architecture: LangGraph Multi-Agent + Map-Reduce

This document outlines the step-by-step execution lifecycle of the Tracker_Project, detailing how data propagates from user input through the autonomous LangGraph cyclic network, culminating in the final executive output.

Phase 1: Ingestion & Orchestration

1. User Input (FastAPI)

The user submits a list of target competitors via a web interface or API request (e.g., ["Stripe", "Adyen"]). The FastAPI backend intercepts this request.



2. Map-Reduce Initialization

The system loads the `MasterComparisonState`. It spawns completely isolated, parallel LangGraph environments for each requested company to prevent context window pollution and optimize latency.

Phase 2: The Cyclic Autonomous Loop (Per Company)

Inside each isolated graph environment, the agents execute a stateful loop managed by the PostgreSQL checkpointer.

LANGGRAPH CYCLIC NETWORK

3. Supervisor Node (The Router)

Input: Current AgentState

Action: Uses Claude 3.5 / GPT-4o with strict Pydantic structures to evaluate the conversation history. It deterministically routes the flow to the next required agent.

Output Decision: Search, Analyze, Write, or FINISH.



4a. Scout Node (Data Fetching)

If routed here, the Scout uses the Tavily API to scrape live news, product updates, and pricing. It injects the raw findings into `scouted_data` and loops back to the Supervisor.

↑ ↓ (Loops back to Supervisor)

4b. Brain Node (RAG & SLM)

If routed here, it utilizes the FinancialBERT SLM to generate embeddings for the `scouted_data`, queries Qdrant/ChromaDB for historical changes, classifies sentiment, and updates `historical_context`. Loops back to the Supervisor.

↑ ↓ (Loops back to Supervisor)

4c. Writer Node (Synthesis)

Digests all state variables to draft the markdown report. **Crucial:** If data is insufficient, it modifies the state to trigger the Supervisor to route back to the Scout. Otherwise, saves to `final_report`.

Phase 3: Verification & Output

5. Human-In-The-Loop (HITL) Interrupt

Before finalizing, the graph hits `interrupt_before=["publish_report"]`. Execution pauses entirely. The state is serialized via `PostgresSaver`. A notification is sent to the operator (e.g., Evitamin account manager) to approve or reject the draft.



6. Reduce Phase (Consolidation)

Once all parallel graphs successfully complete and pass human review, the master node aggregates the individual reports into the final **Competitive Landscape Comparison Matrix**.



7. Final Delivery

The system returns the finalized intelligence report (as PDF or JSON) back to the user through the FastAPI response payload.